DAY 3

Strings, Lists and Sets

4-page quick summary | for students new to Python

The big idea: pick the right container. Use a **str** for text, a **list** for an ordered collection that changes, and a **set** for unique items.

`str "text"`	`list [ ]`	`set { }`
ordered | cannot be changed	ordered | changeable | duplicates OK	unordered | unique items only

1 Strings (str): text you can slice and clean

Text in single, double or triple quotes. Strings are **immutable**: methods return a **new** string, the original never changes.

character	P	Y	T	H	O	N
index	0	1	2	3	4	5
negative	-6	-5	-4	-3	-2	-1

- Index starts at **0**; **-1** is the last character.
- Slice `[start:stop]`: the stop is **excluded**.
- `[::-1]` reverses the text.
- Lists use the same indexing and slicing.

```
word = "PYTHON"
print(word[0], word[-1])
print(word[0:3])
print(word[2:])
print(word[2:4])
print(word[::-1])
```

```
OUTPUT
P N
PYT
THON
TH
NOHTYP
```

Useful string methods

Method	Purpose	Example	Result
`.lower() / .upper()`	all small / all capitals	`"Python".upper()`	`"PYTHON"`
`.title()`	capitalise each word	`"rahul sharma".title()`	`"Rahul Sharma"`
`.capitalize()`	only the first character	`"hello world".capitalize()`	`"Hello world"`
`.strip()/.lstrip()/.rstrip()`	trim both / left / right	`" hi ".strip()`	`"hi"`
`.replace(old, new)`	replace matching text	`"cat".replace("c", "b")`	`"bat"`
`.split()`	text into a **list**	`"red blue".split()`	`['red', 'blue']`
`"sep".join(list)`	list into text	`", ".join(["red", "blue"])`	`"red, blue"`
`.find(text)`	first index, or -1	`"Python".find("th")`	`2`
`.startswith() / .endswith()`	check the start / end	`"a@b.com".endswith(".com")`	`True`

REMEMBER Matching is **case-sensitive**, so normalise with `.lower()` first. Methods return a new string, so **save it**: `name = name.strip()`.

Check what is inside a string (True / False)

Method	True when the text is...	Example	Result
`.isalpha()`	letters only	`"Python".isalpha()`	`True`
`.isdigit()`	digits only	`"12345".isdigit()`	`True`
`.isalnum()`	letters and digits	`"Python123".isalnum()`	`True`
`.isspace()`	whitespace only	`" ".isspace()`	`True`
`.islower()` / `.isupper()`	all small / all capitals	`"hello".islower()`	`True`

Combine strings and format output

```
first_name = "Priya"
last_name = "Patil"
marks = 92.678
full_name = first_name + " " + last_name
print(f"Student: {full_name}")
print(f"Marks: {marks:.2f}")
print(f"Percentage: {0.92678:.1%}")
```

```
OUTPUT
Student: Priya Patil
Marks: 92.68
Percentage: 92.7%
```

2 Lists (list): the most used collection

A list holds many values in one variable, written with **[ ]**. Lists are **ordered**, **mutable** (changeable) and allow **duplicates**. An empty list is `[]`.

```
students = ["Rahul", "Priya", "Amit", "Priya"]
print(students[0], students[-1])
print(students[:-2])
print(students[::-1])
print(len(students))

marks = [78, 92, 85]
marks[0] = 80
print(marks)
```

```
OUTPUT
Rahul Priya
['Rahul', 'Priya']
['Priya', 'Amit', 'Priya', 'Rahul']
4

[80, 92, 85]
```

Add items

Each example starts from `students = ["Rahul", "Priya"]`.

Method	What it does	Result
`.append("Amit")`	adds **one** item at the end	`['Rahul', 'Priya', 'Amit']`
`.insert(1, "Ram")`	adds at index 1; the rest shift right	`['Rahul', 'Ram', 'Priya']`
`.extend(["Isha", "Kabir"])`	adds **each** item of another list	`['Rahul', 'Priya', 'Isha', 'Kabir']`

Remove items

Each example starts from `items = ["pen", "book", "pencil", "book"]`.

Method	What it does	Result
`.remove("book")`	removes the **first** match only	`['pen', 'pencil', 'book']`
`.pop()`	removes and returns the **last** item	`returns 'book'; ['pen', 'book', 'pencil']`
`.pop(1)`	removes and returns the item at index 1	`returns 'book'; ['pen', 'pencil', 'book']`
`del items[0]`	deletes by index (or a slice)	`['book', 'pencil', 'book']`
`.clear()`	empties the list (use with caution)	`[]`

Sort, reverse and copy

Method	What it does
`sort()`	sorts the **original** list in place (returns None)
`sorted(list)`	returns a **new** sorted list; the original stays the same
`reverse()`	reverses the original list in place
`copy()`	makes a separate (shallow) copy, so changing the copy leaves the original alone

```
marks = [85, 72, 96, 81]
print(sorted(marks))
print(marks)
marks.sort()
backup = marks.copy()
backup.append(100)
print(marks, backup)
```

```
OUTPUT
[72, 81, 85, 96]
[85, 72, 96, 81]
[72, 81, 85, 96] [72, 81, 85, 96, 100]
```

Useful functions and membership

```
marks = [78, 92, 85, 88]
print(len(marks), min(marks), max(marks))
print(sum(marks), sum(marks) / len(marks))
print(70 in marks)
```

```
OUTPUT
4 78 92
343 85.75
False
```

3 Sets (set): unique items only

A set stores **unique** items. It is mutable, has **no indexing** and no guaranteed order (printed order may differ). Make one with **{ }** or `set()`.

```
subjects = {"Python", "Java", "Excel", "Python"}
print(subjects)
print("Java" in subjects)
print(len(subjects))
```

```
OUTPUT
{'Excel', 'Python', 'Java'}
True
3
```

WATCH OUT `{}` creates an empty **dict**, not a set. Use `set()` for an empty set: `type({})` gives dict, `type(set())` gives set.

Add and remove set items

Method	What it does
`add(item)`	adds one item
`update(iterable)`	adds many items (duplicates are ignored)
`discard(item)`	removes the item; **no error** if it is missing
`remove(item)`	removes the item; raises **KeyError** if it is missing
`pop() / clear()`	removes an arbitrary item / empties the set

Set operations

Operation	Operator	Meaning	A = {1, 2, 3}, B = {3, 4, 5}
Union	`A | B`	items in A **or** B	`{1, 2, 3, 4, 5}`
Intersection	`A & B`	items in **both**	`{3}`
Difference	`A - B`	items in A but **not** B	`{1, 2}`

```
A = {1, 2, 3, 6}
B = {3, 4, 5, 6}
print(A | B, A & B)
print(A - B, B - A)

names = ["Priya", "Amit", "Priya", "Rahul"]
print(sorted(set(names)))
```

```
OUTPUT
{1, 2, 3, 4, 5, 6} {3, 6}
{1, 2} {4, 5}

['Amit', 'Priya', 'Rahul']
```

Remove duplicates: `set(names)` keeps each name once. Converting back with `list()` does not promise the original order, so use `sorted()` for a tidy order.

4 Choosing the right type

Type	Ordered?	Mutable?	Duplicates?	Best for
`str`	Yes	No	characters may repeat	text
`list`	Yes	Yes	Yes	a changeable sequence
`set`	no positional order	Yes	No	unique values and set operations

RECAP Strings are for **text**, lists are for **ordered collections that may change**, and sets are for **unique items and comparisons**.

5 Mistakes, practice and recap

Spot the bug

Buggy line	Problem	Fix
`word[0] = "J"`	TypeError: strings are immutable	`word = "J" + word[1:]`
`marks.append(4, 5)`	TypeError: append() takes one item	`marks.extend([4, 5])`
`new = marks.sort()`	new is None: sort() works in place	`new = sorted(marks)`
`skills.remove("Go")`	KeyError if "Go" is not in the set	`skills.discard("Go")`
`empty = {}`	this is a dict, not a set	`empty = set()`

Practice questions: model solutions

Your code may differ and still be right. Compare the **idea**, not the wording.

Strings 1-2: title case; strip + upper

```
print(input("Name: ").strip().title())
print("  Python for Beginners  ".strip().upper())
```

Lists 1-2: create; first and last

```
subjects = ["Maths", "Python", "SQL", "Art", "Music"]
print(subjects[0], subjects[-1])
```

List 5: highest, lowest, total, average

```
marks = [78, 92, 85, 66, 91]
print(max(marks), min(marks), sum(marks))
print(sum(marks) / len(marks))
```

Sets 3-5: both, either, Python only

```
python_students = {"Rahul", "Priya", "Amit"}
sql_students = {"Priya", "Neha", "Amit"}
print(python_students & sql_students)
print(python_students | sql_students)
print(python_students - sql_students)
```

Strings 3-5: replace, split, endswith

```
s = "Computer Science"
print(s.replace("Science", "Programming"))
print("Python,SQL,Excel".split(","))
print("asha@mail.com".endswith(".com"))
```

Lists 3-4: append, insert, remove

```
subjects.append("Excel")
subjects.insert(1, "Git")
subjects.remove("Art")
print(subjects)
```

Sets 1-2: unique, add, discard

```
subjects = {"Maths", "Python", "SQL", "Python"}
subjects.add("Art")
subjects.discard("SQL")
print(subjects)
```

Homework Tasks 1-10: topics

1 profile with strings | 2 .upper() .lower() .title() | 3 .strip() | 4 .replace() | 5 .split() | 6 lists and indexing | 7 append() insert() | 8 remove() pop() | 9 sort() len() | 10 mini challenge: strings + lists

Learning checklist: tick each box when you can do it confidently

- [ ] Create and manipulate strings: indexing, slicing and methods.
- [ ] Access, slice, update, add and remove list items.
- [ ] Use common list methods and built-in functions: len(), min(), max(), sum().
- [ ] Create sets to store unique values.
- [ ] Perform union, intersection, difference and membership checks.